Technische Universität Berlin
Faculty IV: Electrical Engineering and Computer Science
Freie Universität Berlin
Department of Mathematics and Computer Science
Algorithmic Bioinformatics

welche uni angeben, welche reihenfolge..?

Automatic Optimization of Level Parameters for RSHash

(RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers. It was recently introduced by the Reinert Group.)

Bjarne David Jehn

Thesis submitted in fulfillment of the requirements for the degree "Bachelor of Science"(B. Sc.) in the field of Computer Science

First Reviewer:	Prof. Dr. Knut Reinert
Second Reviewer:	Prof. Dr. Stephan Kreutzer
Supervisor:	Jonas Schulte-Mattler

Declaration of Authorship

I hereby declare that the thesis submitted is my own, unaided work, completed without any external help. Only the sources and resources listed were used. All passages taken from the sources and aids used, either unchanged or paraphrased, have been marked as such.

Where generative AI tools were used, I have indicated the product name, manufacturer, the software version used, as well as the respective purpose (e.g. checking and improving language in the texts, systematic research). I am fully responsible for the selection, adoption, and all results of the AI-generated output I use.

I have taken note of the Principles for Ensuring Good Research Practice at TU Berlin dated 15 February 2023. https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMBl/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf

I further declare that I have not submitted the thesis in the same or similar form to any other examination authority.

Berlin, _____ _____

Date Signature

Abstract

in English

Zusammenfassung

auf Deutsch

Contents

1 Introduction **5**
- 1.1 Notation . . . 5
- 1.2 RSHash Explanation . . . 5
- 1.3 Problem Definition . . . 6

2 Sequence Statistics **7**
- 2.1 Minimizer Distribution . . . 7
- 2.2 Average Frequencies $f_{avg}(m,t)$ . . . 8
- 2.3 Times for Operations on Datastructure . . . 8

3 MINLP Modeling **9**
- 3.1 Model Input . . . 9
- 3.2 Model for One Level . . . 9
  - 3.2.1 Lookup Time . . . 10
  - 3.2.2 Space Consumption . . . 11
  - 3.2.3 Optimization Goal . . . 13
- 3.3 Model for Several Levels . . . 15
  - 3.3.1 Lookup Time . . . 16
  - 3.3.2 Space Consumption . . . 16
- 3.4 Limitations . . . 17

4 Subsampling **18**
- 4.1 Motivation . . . 18
- 4.2 Subsampling Method . . . 18
- 4.3 Time Complexity of Finding the Minimizer Distributions . . . 18
- 4.4 Time Complexity using Subsampling . . . 18
- 4.5 Error Analysis . . . 19
- 4.6 Choosing l . . . 19
- 4.7 Choosing n . . . 19

5 Experiments **20**
- 5.1 Solving the MINLP . . . 20

Bibliography **21**

1 Introduction

1.1 Notation

Let q be a k-mer. We refer to its minimizer $m(q)$ and the minimizers frequency $f(m(q))$ i.e. the number of occurrences of the minimizer in a given set of input sequences. For a threshold t call a minimizer *infrequent* only if $1 \leq f(m(q)) \leq t$ holds i.e. the minimizer occurs at least once but no more than the given threshold.

define super-k-mer

1.2 RSHash Explanation

Wie detailiert soll ich RSHash beschreiben? Reicht eine grobe Beschreibung und ein Verweis auf [SMR26]?

RSHash is a minimizer based fast, compressed hash table for storing DNA k-mers that answers lookup and can be extended for supporting locate. It was recently introduced by Schulte-Mattler and Reinert [SMR26].
The data structure works with one, two or three levels and a hash table. For each level l a pair of level parameters has to be specified: the minimizer length m^l and a threshold t^l.

Each level l is build in the following way:
For each k-mer k in the input sequences the minimizer $m^l(k)$ of length m^l is computed. A temporary hash table is used to count the number of occurrences of each appearing minimizer, also referred to as the minimizers frequency $f(m^l(k))$.
A bit vector R^l of lengths 4^{m^l} is used to mark all minimizers that appear at least once but no more than t^l times. We call those minimizers *infrequent*.
The position of every occurrence of each infrequent minimizer in the text are stored in the offset vector O^l in the order induced by R^l.
The boundaries of offsets in O^l belonging to different minimizers are marked in the bit vector $S^l \in \{0,1\}^{|O|+1}$.
Those k-mers with frequent minimizers on level l make up the input sequences for the next higher level $l+1$ which works exactly like the previous apart from having its own level parameters. K-mers with frequent minimizers on all specified levels end up being stored in a hash table H.

1.3 Problem Definition

Choosing the minimizer length m is an inherent problem of minimizer based k-mer indexing methods. Zheng, Kingsford, and Marçais [ZKM20] show that a minimizer is expected to be valid for an average of $(k - m + 2)/2$ k-mers. Using a small $m << k$ thus leads to a relatively small number of super-k-mers meaning a better compression. Picking a small m however also means that minimizers will be more frequent which leads to high query times because each minimizer offset has to be evaluated for an actual match with the queried k-mer. This means a constant trade off between space consumption and query times. Existing methods use a number of approaches to deal with this problem

beispiele und quellen angeben

The leveled approach used by RSHash exploits the skewed distribution of minimizers. It allows to achieve high compression by using relatively short minimizers on the lower levels whilst also mitigating the burden of very frequent minimizers by using thresholds to move them to higher levels.

So far the level parameters have to be specified when building the data structure which requires manual testing and extensive understanding of the RSHash algorithm.

With this thesis we attempt to expand the understanding of how exactly the level parameters influence the time and space consumption of RSHash and make the algorithm more accessible to use by automating the optimization of level parameters for a given genome and optimization goal.

Problems:

- work towards a better understanding of minimizer distribution across several levels
- analyze how level parameters influence time and space consumption of RSHash
- automize finding optimal level parameters

2 Sequence Statistics

2.1 Minimizer Distribution

The skewness of minimizer distributions has been observed by several previous works [Chi+14; Jai+20; Pib22; SMR26].
We will focus on the percentages $p_M \in [0,1]$ and $p_K \in [0,1]$ as introduced by Schulte-Mattler and Reinert [SMR26]:

Consider a fixed sequence S, minimizer length m and threshold t:
Let $\mathbf{M}$ be the multiset of minimizers in S and $\mathbf{M}_{infrequent} \subseteq \mathbf{M}$ be the submultiset of those minimizers that are *infrequent*. It is important to note that the multisets contain each minimizers as many times as there are super-k-mers sk in the sequence which have that minimizer:

$$\mathbf{M} := \{m(sk) \mid \forall sk \in S\}$$
$$\mathbf{M}_{infrequent} := \{m(sk) \mid \forall sk \in S.f(m(sk)) \leq t\} \subseteq \mathbf{M}$$

Then $p_M(m,t)$ describes how likely an occurring minimizer is to be *infrequent*:

$$p_M(m,t) = \frac{|\mathbf{M}_{infrequent}|}{|\mathbf{M}|} \in [0,1]$$

Let $\mathbf{K}$ be the multiset of all k-mers k in the sequence and $\mathbf{K}_{infrequent} \subseteq \mathbf{K}$ be the multiset of all k-mers with *infrequent* minimizers:

$$\mathbf{K} := \{k \forall k \in S\}$$
$$\mathbf{K}_{infrequent} := \{k \mid \forall k \in S.f(m(k)) \leq t\} \subseteq \mathbf{K}$$

Then $p_K(m,t)$ describes how likely a k-mer in the sequence is to have an *infrequent* minimizer:

$$p_K(m,t) = \frac{|\mathbf{K}_{infrequent}|}{|\mathbf{K}|} \in [0,1]$$

We aim to analyze how these two percentages describing the minimizer distribution behave for several levels as used by RSHash in order to gain a better understanding of how to chose the level parameters.

The distribution of minimizers on higher levels should be highly dependent on the choice of the level parameters off the levels below, as those influence which k-mers appear on higher levels.
In order to understand this relationship, we will measure these distributions for a selection of sequences and analyze the results. It should be noted that this alone is not sufficient to argue about the general case.

2.2 Average Frequencies $f_{avg}(m,t)$

Let k be a k-mer with an *infrequent* minimizer $m(k)$. The lookup of k is completed by a search for k in T at every offset of $m(k)$ in the offset vector O. In order to model the lookups runtime in Chapter 3, we introduce the value $f_{avg}(m,t)$ to describe the average frequency of $m(k)$ and thus the average amount of relevant offsets in O when looking up k whilst using minimizer length m and threshold t..
We recall that $p_K(m,t)$ is the percentage of k-mers with *infrequent* minimizers when using minimizer length m and threshold t. Then $(p_K(m,f) - p_K(m,f-1))/p_K(m,t)$[1] percent of those k-mers with *infrequent* minimizers have minimizers of exact frequency $f(m(k)) = f$.
We can derive $f_{avg}(m,t)$ from $p_K(m,t)$ by computing a sum over every frequency $1 \leq f \leq t$ weighed with the percentage of *infrequent* k-mers with that frequency:

$$f_{avg}(m,t) = \frac{1}{p_K(m,t)} \cdot \sum_{f=1}^{t} (p_K(m,f) - p_K(m,f-1)) \cdot f$$

tabelle $f_avg(m,t)$ mittels dynamischer programmierung bauen? - eigentlich nicht nötig, wenn distinct stats (also nicht die cumulativen verwendet werden)

2.3 Times for Operations on Datastructure

We denote the time needed to perform an operation *op* on a data structure *struct* with $T_{op_{struct}}$ e.g. the time for a rank query on the bit vector R is T_{rank_R}.

incomplete. measure or model?

Vector Access Accessing an element of a Vector can be done in constant time, we use the following values based on measurements:

$$T_{acc_R} = ...$$
$$T_{acc_O} = ...$$

Hash Table Access Accessing an element of a Hash Table can be done in constant time, we use the following values based on measurements:

$$T_{lookup_H} = ...$$

Bit Vector Rank Query A bitvector of length U with n 1s can be encoded with $n\lceil log_2 U/n \rceil + 3n$ many bits supporting rank in $O(log_2(U/n)$ time (Okanohara and Sadakane, 2007).

does that help here? Doesn't the model require actual time instead of O notation - maybe measurements would be more suited

Bit Vector Select Query ???
Comparison with Text ???

[1] Note that $p_K(m,0) = 0$ per definition

3 MINLP Modeling

We model the optimization problem as a Mixed Integer Non Linear Programming (MINLP). In Section 3.2 we model the runtime and space needed by RSHash on one level in dependence of the level parameters and minimizers distribution. We then extend this to the case of several levels in Section 3.3.
In Chapter 5 we describe how to find the optimal level parameters for a given sequence using the MINLP.

3.1 Model Input

The sets of minimizer lengths $\mathcal{M} \in \mathbb{N}$ and thresholds $\mathcal{T} \in \mathbb{N}$ to consider, the computed minimizer statistics $p_K(m,t) \forall (m,t) \in \mathcal{M} \times \mathcal{T}$ and $f_{avg}(m,t) \forall (m,t) \in \mathcal{M} \times \mathcal{T}$ as well as the measured times $T_{acc_R}, T_{rank_R}, T_{sel_S}, T_{acc_O}, T_{comp_T}, T_{lookup_H}$ needed for the access, rank and select queries on the data structures different components are passed as input to the MINLP. The computation of these values is outlined in Chapter 2.

3.2 Model for One Level

In order to determine $\mathbf{m}$ and $\mathbf{t}$ we use binary indicator variables μ_m and τ_t for each possible $m \in \mathcal{M}$ and $t \in \mathcal{T}$:

$$\mu_m \in \{0,1\} \mid \forall m \in \mathcal{M} \tag{3.1}$$
$$\tau_t \in \{0,1\} \mid \forall t \in \mathcal{T} \tag{3.2}$$

The indicator variables for each minimizer length should be mutually exclusive as $\mathbf{m}$ can only take one distinct value. The same holds for the indicator variables for each threshold:

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \tag{3.3}$$
$$\sum_{t \in \mathcal{T}} \tau_t = 1 \tag{3.4}$$

The variables $\mathbf{m} \in \mathcal{M}$ and $\mathbf{t} \in \mathcal{T}$ are introduced to describe the optimal minimizer length and threshold.
There values can be modeled through a sum over the indicator variables multiplied with

there corresponding values:

$$\mathbf{m} \in \mathcal{M} \tag{3.5}$$
$$\mathbf{t} \in \mathcal{T} \tag{3.6}$$
$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \tag{3.7}$$
$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \tag{3.8}$$

We introduce the variable $\mathbf{p_K} \in [0, 1]$ to take the value of $p_K(\mathbf{m}, \mathbf{t})$ (Section 2.1). As it is not possible to directly evaluate this value as both $\mathbf{m}$ and $\mathbf{t}$ are variables of the model. Instead we use the indicator variables to model a constraint forcing $\mathbf{p_K}$ to take the value of the only $p_K(m, t)$ for which $\mu_m = 1$ and $\tau_t = 1$:

$$\mathbf{p_K} \in [0, 1] \tag{3.9}$$
$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \mu_m \cdot \sum_{t \in \mathcal{T}} \tau_t \cdot p_K(m, t) \tag{3.10}$$

Analogously we introduce $\mathbf{f_{avg}} \in \mathbb{R}_0^+$ to represent $f_{avg}(\mathbf{m}, \mathbf{t})$ (Section 2.2):

$$\mathbf{f_{avg}} \in \mathbb{R}_0^+ \tag{3.11}$$
$$\mathbf{f_{avg}} = \sum_{m \in \mathcal{M}} \mu_m \cdot \sum_{t \in \mathcal{T}} \tau_t \cdot f_{avg}(m, t) \tag{3.12}$$

3.2.1 Lookup Time

In order to model the time needed for a lookup using RSHash, we closely follow the lookup procedure as described by Schulte-Mattler and Reinert [SMR26]:
A k-mer q is looked up by first computing its minimizer $m(q)$ and checking whether the bit $R[m(q)]$ is set, taking T_{acc_R} time (part of the model input Section 3.1).
For the percentage $\mathbf{p_K}$ (Equation (3.10)) of the k-mers this bit is set and the lookup is completed by searching for q on the level. We introduce the variable $T_{level} \in \mathbb{R}_0^+$ to describe the time needed for this.
If the bit $R[m(q)]$ is not set, the lookup is completed through a lookup in the hash table H taking T_{lookup_H} time (part of the model input Section 3.1). The variable $T_{lookup_{RSHash}} \in \mathbb{R}_0^+$ describing the lookup time can thus be modeled as:

$$T_{level} \in \mathbb{R}_0^+ \tag{3.13}$$
$$T_{lookup_{RSHash}} \in \mathbb{R}_0^+ \tag{3.14}$$
$$T_{lookup_{RSHash}} = T_{acc_R} + \mathbf{p_K} \cdot T_{level} + (1 - \mathbf{p_K}) \cdot T_{lookup_H} \tag{3.15}$$

The search for those k-mers q for which the bit $R[m(q)]$ is set continues on the level by first computing the minimizer's rank $r = rank_R(m(q))$ taking T_{rank_R} time.

This rank is then used in the select queries $first = sel_S(r)$ and $last = sel_S(r+1) - 1$ which tell the indices in Offsets O where the minimizer's first and last offset in the text T are stored. These two queries take $2T_{sel_S}$ time.
Finally the presence of q in T is checked at every minimizer offset in the interval $O[first : last]$ until q is either found or the end of the interval is reached. We introduce the variable $\mathbf{f_{avg}} \in \mathbb{R}_0^+$ to describe the average frequency of *infrequent* minimizers and thus the average length of this interval. In the lookup case, the comparison is terminated as soon as q is found for the first time, which is on average the case after $(\mathbf{f_{avg}} + 1)/2$ comparisons (+1 is necessary because the first index always has to be checked) witch each comparison taking T_{comp_T} time.
The following constraint models the complete time T_{level} needed to search for a k-mers presence on the level:

$$T_{level} = T_{rank_R} + 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{acc_O} + T_{comp_T}) \tag{3.16}$$

Without further constraints, solving the described MINLP for minimal $T_{lookup_{RSHash}}$ results in a solution where the μ_m, τ_t corresponding to the smallest $m \in \mathcal{M}$, $t \in \mathcal{T}$ will be set to one.
This is expected, as this leads to the smallest possible $\mathbf{p_K}$ which minimizes the k-mers having to be looked up in the text and maximizes the lookups in the (faster) hash table. This however results in maximal space consumption, as each entry in the hash table is significantly larger than an entry in O as described in Section 1.3.
In order to account for that, we have to add constraints modeling the space consumption.

3.2.2 Space Consumption

Instead of modeling the total space used by RSHash we will focus on the bits per k-mer (bpk) used by the data structure as this metric allows to compare compression independent of the sequence length.
The space consumption of RSHash can be decomposed into the space needed for the data structures components: the text T, the vectors R, S and O as well as the hash table H. We use one variable to describe the bits per k-mer needed by each of the components:

$$bpk_T \in \mathbb{R}_0^+ \tag{3.17}$$
$$bpk_R \in \mathbb{R}_0^+ \tag{3.18}$$
$$bpk_S \in \mathbb{R}_0^+ \tag{3.19}$$
$$bpk_O \in \mathbb{R}_0^+ \tag{3.20}$$
$$bpk_H \in \mathbb{R}_0^+ \tag{3.21}$$

The text T is much longer than k. Because of this the bits per k-mer for storing the text T can be set to a constant of 2 bits needed to store one base:

$$bpk_T = 2 \tag{3.22}$$

Bit vector R is a bit vector of length $4^{\mathbf{m}}$ in which each bit set to 1 marks the occurrence of one distinct *infrequent* minimizer in the sequences. The number of distinct *infrequent* minimizers can be computed by dividing the number of k-mers with infrequent minimizers through the average frequency of their minimizers: $\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}$. Using Elias-Fano-Encoding, its size can be compressed to $\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \lceil log_2(4^{\mathbf{m}} / \frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}) \rceil + 3\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}$ [OS07] . Dividing this by the total number n_{kmers} of k-mers yields the constraint describing the bits per k-mer of R:

make n_{kmers} part of model

woher kommt diese formel, kann sie so im paper nicht finden

$$
\begin{aligned}
bpk_R &= \frac{1}{n_{kmers}} \cdot \left(\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \lceil log_2 \frac{4^{\mathbf{m}}}{\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}} \rceil + 3\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \right) \\
&= \frac{\mathbf{p_K}}{\mathbf{f_{avg}}} \lceil log_2 \frac{4^{\mathbf{m}}}{\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}} \rceil + 3\frac{\mathbf{p_K}}{\mathbf{f_{avg}}} \\
&= \frac{\mathbf{p_K}}{\mathbf{f_{avg}}} \left(\lceil log_2 \frac{4^{\mathbf{m}}}{\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}} \rceil + 3 \right) \\
&= \frac{\mathbf{p_K}}{\mathbf{f_{avg}}} \left(\lceil log_2 4^{\mathbf{m}} - log_2 \frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \rceil + 3 \right) \\
&= \frac{\mathbf{p_K}}{\mathbf{f_{avg}}} \left(\lceil 2\mathbf{m} - log_2 \frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \rceil + 3 \right)
\end{aligned}
\tag{3.23}
$$

Bit vector S has length $|O| + 1 \approx |O|$ where every bit corresponds to the offset of one minimizer encountered in the sequences and one extra bit marks the end of all offsets - this last bit can however be disregarded when modeling bpk_S as its effect is insignificantly small.
Since $\mathbf{p_K}$ describes the percentage of k-mers with *infrequent* minimizers and every minimizer covers an average of $(k - \mathbf{m} + 2)/2$ consecutive k-mers, we can model the bits per k-mer in S through:

$$
bpk_S = \frac{\mathbf{p_K}}{(k - \mathbf{m} + 2)/2} \tag{3.24}
$$

The offset vector O contains one entry of size $log_2|T|$ for the offset of each *infrequent* minimizer.
Since bpk_S already describes the number of offsets per k-mer, we can model the bits per k-mer in O with:

$$
bpk_O = bpk_S \cdot log_2|T| \tag{3.25}
$$

The hash table H contains an entry $(key : q, value : offset(q))$ for $1 - \mathbf{p_K}$ of the k-mers q in T. Each entry has size $size(key) + size(value) = 64 + log_2|T|$. We introduce the load factor α to the model, in order to account for empty buckets in the hash table. The bits per k-mer required for the hash table can thus be modeled by:

reminder to include somewhere that we set $\alpha = 0.75$ in practice and explain why that is a rea-

$$
bpk_H = (1 - \mathbf{p_K}) \cdot \frac{(64 + log_2|T|)}{\alpha} \tag{3.26}
$$

nur hashset für lookup, 1 statt log2T

The total bits per k-mer $bpk_{total} \in \mathbb{R}_0^+$ can be modeled by simply adding up the space requirements of each component:

$$bpk_{total} \in \mathbb{R}_0^+ \tag{3.27}$$
$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \tag{3.28}$$

3.2.3 Optimization Goal

Having defined all constraints regarding the lookup time and space consumption of RSHash, we can now construct a MINLP in order to find optimal level parameters.
We introduce three more external parameters to the model: The optimization goal $OptGoal$ which specifies whether to minimize $T_{lookup_{RSHash}}$ or bpk_{total} and upper bounds $Bound_{time}, Bound_{bpk}$ for the lookup time and space consumption.

$$\begin{aligned} OptGoal &\in \{T_{lookup_{RSHash}}, bpk_{total}\} \\ Bound_{time} &\in \mathbb{R}_0^+ \\ Bound_{bpk} &\in \mathbb{R}_0^+ \end{aligned}$$

When using the MINLP, the upper bound belonging to the optimization goal should be set to ∞. In addition to the already defined ones, the following two constraints are added to the MINLP to enforce these bounds:

$$T_{lookup_{RSHash}} \leq Bound_{time} \tag{3.29}$$
$$bpk_{total} \leq Bound_{bpk} \tag{3.30}$$

One could for example search for level parameters minimizing the lookup time whilst not exceeding eight bits per k-mer by setting $OptGoal \leftarrow T_{lookup_{RSHash}}, Bound_{time} \leftarrow \infty, Bound_{bpk} \leftarrow 8$. The constructed MINLP for this example is:

$$\begin{aligned} INPUT : OptGoal = T_{lookup_{RSHash}}, Bound_{time} = \infty, Bound_{bpk} = 8, \\ p_K(m,t) \forall (m,t) \in \mathcal{M} \times \mathcal{T}, \mathbf{f_{avg}}(m,t) \forall (m,t) \in \mathcal{M} \times \mathcal{T}, \\ T_{acc_R}, T_{rank_R}, T_{sel_S}, T_{acc_O}, T_{comp_T}, T_{lookup_H} \end{aligned}$$

MINIMIZE

$$OptGoal = T_{lookup_{RSHash}}$$

SUBJECT TO

$$\sum_{m \in \mathcal{M}} \mu_m = 1 \qquad \textit{Equation (3.3)}$$

$$\sum_{t \in \mathcal{T}} \tau_t = 1 \qquad \textit{Equation (3.4)}$$

$$\mathbf{m} = \sum_{m \in \mathcal{M}} \mu_m \cdot m \qquad \textit{Equation (3.7)}$$

$$\mathbf{t} = \sum_{t \in \mathcal{T}} \tau_t \cdot t \qquad \textit{Equation (3.8)}$$

$$\mathbf{p_K} = \sum_{m \in \mathcal{M}} \mu_m \cdot \sum_{t \in \mathcal{T}} \tau_t \cdot p_K(m,t) \qquad \textit{Equation (3.10)}$$

$$\mathbf{f_{avg}} = \sum_{m \in \mathcal{M}} \mu_m \cdot \sum_{t \in \mathcal{T}} \tau_t \cdot f_{avg}(m,t) \qquad \textit{Equation (3.12)}$$

$$T_{level} = T_{rank_R} + 2T_{sel_S} + (\mathbf{f_{avg}} + 1)/2 \cdot (T_{acc_O} + T_{comp_T}) \qquad \textit{Equation (3.16)}$$

$$T_{lookup_{RSHash}} = T_{acc_R} + \mathbf{p_K} \cdot T_{level} + (1 - \mathbf{p_K}) \cdot T_{lookup_H} \qquad \textit{Equation (3.15)}$$

$$T_{lookup_{RSHash}} \leq Bound_{time} = \infty \qquad \textit{Equation (3.29)}$$

$$bpk_T = 2 \qquad \textit{Equation (3.22)}$$

$$bpk_R = \frac{1}{n_{kmers}} \cdot \left(\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \lceil log_2 \frac{4^{\mathbf{m}}}{\frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}}} \rceil + 3 \frac{\mathbf{p_K} * n_{kmers}}{\mathbf{f_{avg}}} \right) \qquad \textit{Equation (3.23)}$$

$$bpk_S = \frac{\mathbf{p_K}}{(k - \mathbf{m} + 2)/2} \qquad \textit{Equation (3.24)}$$

$$bpk_O = bpk_S \cdot log_2|T| \qquad \textit{Equation (3.25)}$$

$$bpk_H = (1 - \mathbf{p_K}) \cdot \frac{(64 + log_2|T|)}{\alpha} \qquad \textit{Equation (3.26)}$$

$$bpk_{total} = bpk_T + bpk_R + bpk_S + bpk_O + bpk_H \qquad \textit{Equation (3.28)}$$

$$bpk_{total} \leq Bound_{bpk} = 8 \qquad \textit{Equation (3.30)}$$

VARIABLES

$$\mu_m \in \{0,1\} \mid \forall m \in M \qquad \textit{Equation (3.1)}$$

$$\tau_t \in \{0,1\} \mid \forall t \in T \qquad \textit{Equation (3.2)}$$

$$\mathbf{m} \in \mathcal{M} \qquad \textit{Equation (3.5)}$$

$$\mathbf{t} \in \mathcal{T} \qquad \textit{Equation (3.6)}$$

$$\mathbf{p_K} \in [0,1] \qquad \textit{Equation (3.9)}$$

$$\mathbf{f_{avg}} \in \mathbb{R}_0^+ \qquad \textit{Equation (3.11)}$$

$$T_{level} \in \mathbb{R}_0^+ \qquad \textit{Equation (3.13)}$$

$$T_{lookup_{RSHash}} \in \mathbb{R}_0^+ \qquad \textit{Equation (3.14)}$$

$$bpk_T \in \mathbb{R}_0^+ \qquad \textit{Equation (3.17)}$$

$$bpk_R \in \mathbb{R}_0^+ \qquad \textit{Equation (3.18)}$$

$$bpk_S \in \mathbb{R}_0^+ \qquad \textit{Equation (3.19)}$$

$$bpk_O \in \mathbb{R}_0^+ \qquad \textit{Equation (3.20)}$$

$$bpk_H \in \mathbb{R}_0^+ \qquad \textit{Equation (3.21)}$$

$$bpk_{total} \in \mathbb{R}_0^+ \qquad \textit{Equation (3.27)}$$

add equation references in MINLP

3.3 Model for Several Levels

In the following, we adapt the constraints defined for using only one level in Section 3.2 to model the use of $L \geq 1$ levels with $\mathcal{L} := \{1, ..., L\}$ being the set of used levels.
When modeling several levels, we add a superscript l to variables in order to indicate there belonging to a level $1 \leq l \leq L$ e.g. $\mathbf{m}^1, \mathbf{m}^2$ and $\mathbf{m}^3$ denote the optimal minimizer lengths on level $1, 2$ and 3.
We still assume, that the times $T_{acc_{R^l}}, T_{rank_{R^l}}, T_{sel_{S^l}}, T_{acc_{O^l}}, T_{comp_T}, T_{lookup_H}$ are (practically) independent of the parameters $\mathbf{m}^l, \mathbf{t}^l$ and the level l. reason

Unique sets of binary indicator variables μ_m^l and τ^l are needed for the level parameters $\mathbf{m}^l$ and $\mathbf{t}^l$ of each level l. The constraints regarding these variables are analog to those when using only one level:

$$
\begin{aligned}
\mu_m^l &\in \{0,1\} && \mid \forall (l,m) \in (\mathcal{L} \times \mathcal{M}) \\
\tau_t^l &\in \{0,1\} && \mid \forall (l,t) \in (\mathcal{L} \times \mathcal{T}) \\
\sum_{m \in \mathcal{M}} \mu_m^l &= 1 && \mid \forall l \in \mathcal{L} \\
\sum_{t \in \mathcal{T}} \tau_t^l &= 1 && \mid \forall l \in \mathcal{L} \\
\mathbf{m}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{t}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{m}^l &= \sum_{m \in \mathcal{M}} \mu_m^l \cdot m^l && \mid \forall l \in \mathcal{L} \\
\mathbf{t}^l &= \sum_{t \in \mathcal{T}} \tau_t^l \cdot t^l && \mid \forall l \in \mathcal{L}
\end{aligned}
$$

Let $\bar{m}^l := [m^i \mid \forall 1 \leq i \leq l]$ be the vector containing a minimizer lengths for each of the current and all previous levels and $\bar{t}^l := [t^i \mid \forall 1 \leq i \leq l]$ the corresponding thresholds.
For each level l $\mathbf{p}_\mathbf{K}^l$ and $\mathbf{f}_\mathbf{avg}^l$ can then be modeled analogously to using only one level by making use of the Section 2.1:

genauer erklären und oder beispiel hinzufügen

$$
\begin{aligned}
\mathbf{p}_\mathbf{K}^l &\in [0,1] && \mid \forall l \in \mathcal{L} \\
\mathbf{f}_\mathbf{avg}^l &\in \mathbb{R}_0^+ && \mid \forall l \in \mathcal{L} \\
\mathbf{p}_\mathbf{K}^l &= \sum_{\bar{m}^l \in \mathcal{M}^l} \left(\prod_{m \in \bar{m}^l} \mu_m^l \right) \cdot \sum_{\bar{t}^l \in \mathcal{T}^l} \left(\prod_{t \in \bar{t}^l} \tau_t^l \right) \cdot p_K^l(\bar{m}^l, \bar{t}^l) && \mid \forall l \in \mathcal{L} \\
\mathbf{f}_\mathbf{avg}^l &= \sum_{\bar{m}^l \in \mathcal{M}^l} \left(\prod_{m \in \bar{m}^l} \mu_m^l \right) \cdot \sum_{\bar{t}^l \in \mathcal{T}^l} \left(\prod_{t \in \bar{t}^l} \tau_t^l \right) \cdot f_{avg}^l(\bar{m}^l, \bar{t}^l) && \mid \forall l \in \mathcal{L}
\end{aligned}
$$

3.3.1 Lookup Time

The search time on a level $l \in \mathcal{L}$ can be leveled analogously to that on one level:

$$T^l_{level} \in \mathbb{R}^+_0 \qquad | \, \forall l \in \mathcal{L}$$

$$T^l_{level} = T_{rank_{R^l}} + 2T_{sel_{S^l}} + \frac{\mathbf{f^l_{avg}} + 1}{2} \cdot (T_{acc_{O^l}} + T_{comp_T}) \qquad | \, \forall l \in \mathcal{L}$$

For $\mathbf{p}^l_\mathbf{K}$ of the k-mers on a level $l \in \mathcal{L}$ the lookup is completed by a search through the offsets on that level whilst the remaining $(1 - \mathbf{p}^l_\mathbf{K})$ of queries result in a lookup on the next level.
We can thus model the lookup time recursively:

$$T^l_{lookup_{RSHash}} = \begin{cases} T_{acc_{R^l}} + \mathbf{p}^l_\mathbf{K} \cdot T^l_{level} \\ \qquad + (1 - \mathbf{p}^l_\mathbf{K}) \cdot T^{l+1}_{lookup_{RSHash}} & | \, l \in \mathcal{L} \\ T_{lookup_H} & | \, l \notin \mathcal{L} \end{cases}$$

As an example for $L = 2$ levels, this results in:

$$\begin{aligned} T^1_{lookup_{RSHash}} &= T_{acc_{R^1}} + \mathbf{p}^1_\mathbf{K} \cdot T^1_{level} \\ &\quad + (1 - \mathbf{p}^1_\mathbf{K}) \cdot \Big(T_{acc_{R^2}} + \mathbf{p}^2_\mathbf{K} \cdot T^2_{level} \\ &\qquad + (1 - \mathbf{p}^2_\mathbf{K}) \cdot T_{lookup_H} \Big) \\ &= T_{acc_{R^1}} + \mathbf{p}^1_\mathbf{K} \cdot (T_{rank_{R^1}} + 2T_{sel_{S^1}} + \frac{\mathbf{f^l_{avg}} + 1}{2} \cdot (T_{acc_{O^1}} + T_{comp_T})) \\ &\quad + (1 - \mathbf{p}^1_\mathbf{K}) \cdot \Bigg(T_{acc_{R^2}} + \mathbf{p}^2_\mathbf{K} \cdot (T_{rank_{R^2}} + 2T_{sel_{S^2}} + \frac{\mathbf{f^2_{avg}} + 1}{2} \cdot (T_{acc_{O^2}} + T_{comp_T})) \\ &\qquad + (1 - \mathbf{p}^2_\mathbf{K}) \cdot T_{lookup_H} \Bigg) \end{aligned}$$

zweiten teil weglassen?

3.3.2 Space Consumption

The text T is only being stored once, so the constant approximation $bpk_T \approx 2$ still holds.

Bit vector R^l is modeled analogously to using only one level:

$$bpk_{R^l} \in \mathbb{R}^+_0 \qquad | \, \forall l \in \mathcal{L}$$

$$bpk_{R^l} = \frac{1}{n^l_{kmers}} \cdot \left(\frac{\mathbf{p}^l_\mathbf{K} * n^l_{kmers}}{\mathbf{f^l_{avg}}} \lceil log_2 \frac{4^\mathbf{m}}{\frac{\mathbf{p}^l_\mathbf{K} * n^l_{kmers}}{\mathbf{f^l_{avg}}}} \rceil + 3 \frac{\mathbf{p}^l_\mathbf{K} * n^l_{kmers}}{\mathbf{f^l_{avg}}} \right) \qquad | \, \forall l \in \mathcal{L}$$

Bit vector S^l on level l has length $|O^l| + 1 \approx |O^l|$. Only k-mers that have frequent minimizers on all previous levels but an *infrequent* minimizer on level l lead to an offset in O^l and thus to a bit in S^l:

$$bpk_{S^l} \in \mathbb{R}_0^+ \qquad | \; \forall l \in \mathcal{L}$$
$$bpk_{S^l} = \left(\prod_{i=1}^{l-1} 1 - \mathbf{p}_\mathbf{K}^i \right) \cdot \frac{\mathbf{p}_\mathbf{K}^l}{(k - \mathbf{m}^l + 2)/2} \qquad | \; \forall l \in \mathcal{L}$$

The offset vector O^l is modeled analogously to using only one level:

$$bpk_{O^l} \in \mathbb{R}_0^+ \qquad | \; \forall l \in \mathcal{L}$$
$$bpk_{O^l} = bpk_{S^l} \cdot log_2|T| \qquad | \; \forall l \in \mathcal{L}$$

The hash table H only contains an entry for k-mers q, which have frequent minimizers on all levels. The bits per k-mer for the hash table can thus be modeled as

$$bpk_H = \left(\prod_{l=1}^{L} 1 - \mathbf{p}_\mathbf{K}^l \right) \cdot \frac{(64 + log_2|T|)}{\alpha}$$

The total bits per k-mer for L levels only differ to those on one level, as there are now a vector R, S and O on each level:

$$bpk_{total} = bpk_T + \sum_{l=1}^{L} (bpk_{R^l} + bpk_{S^l} + bpk_{O^l}) + bpk_H$$

3.4 Limitations

The MINLP approach for finding optimal level parameters has the advantage of finding the optimal level parameters for the specified optimization goal.

However, finding the empirical values $p_K^l(m, t)$ for each level becomes increasingly time consuming for large genomes and more levels used as outlined in Section 2.1. This renders the approach implausible.

noch nicht geschrieben

For this reason, an approach yielding only an approximation of the optimal level parameters but taking less time to do so could be of interest.

We will further explore the method of Chapter 4 the genome in order to reduce the computation of minimizer distribution.

TODO

In another approach we will try to approximate the distributions on higher levels based on that on the first level in order to get arround the exponential cost of computing those values.

TODO

4 Subsampling

4.1 Motivation

The time for processing a genome in order to find the minimizer distribution $\mathbf{p_K}(m, t)$ for possible minimizer lengths $m \in \mathcal{M}$ and thresholds $t \in \mathcal{T}$ is directly proportional to the size of the genome.
For large genomes, this processing can become the dominating factor when searching for optimal parameters using the Chapter 3 approach. In order to speed up the search for the empirical values, we introduce a subsampling method.

4.2 Subsampling Method

Instead of calculating the distribution over the full genome, we sample n sequences of length $l \geq k$ at random positions in the genome.

how to handle overlapping samples

müsste die gezählte frequenz der minimizer mit eine Faktor mutipliziert werden, um auf die länge des ursprünglichen textes zu skalieren?? schließlich kommen minimizer ja in einem kürzeren text perse seltener vor als in einem längeren

4.3 Time Complexity of Finding the Minimizer Distributions

genaue zeitliche abhängigkeit 1. verstehen und 2. aufschreiben

4.4 Time Complexity using Subsampling

By applying the proposed subsampling strategy, the original genome T of length $|T|$ is reduced to the subsampled text T^S of length $|T^S| = n \cdot l$. This means a reduction in size by a factor of $\frac{|T|}{|T^S|}$.
Depending on the choice of n and l this results in a speedup of the processing time by a factor of or even a constant processing time when both parameters are chosen as a constand regardless of $|T|$ (this however should result in a growing error for larger genomes).

assuming no overlaps

factor

time

4.5 Error Analysis

4.6 Choosing l

The length l has to be at least k.

wenn $l > k$ ist, überlappen sich mehrere kmere an den gesampleten stellen.
Frage: ich verstehe noch nicht, warum genau dass von vorteil ist.
Antwort: Weil roling hash - l ca 1000, dann quasi linear

4.7 Choosing n

Choosing a constant n would result in a processing time that would no longer be dependent of the genomes length $|T|$, as the sampled text would have constant size $n \cdot l$. For large genomes, the percentage $\frac{|T^S|}{|T|}$ of the genome represented in the subsample can become arbitrarily small, resulting in growing error.
If instead n is chosen relative to $|T|$, $\frac{|T^S|}{|T|}$ will have a constant value independent of the genomes size. Even though this means, that the time needed, in order to find the minimizer distributions would still be proportional to the genomes size, this should be the preferred option as it results in a bounded error of the approximation.
For a large enough n, the probability of choosing a bad representation becomes highly unlikely.

assuming no overlaps

zu vage, begründen

5 Experiments

5.1 Solving the MINLP

For a given sequence, we can compute the minimizer distribution as outlined in Chapter 2 and use a MINLP solver like SCIP [Ach+08; Ach09; Bes+23] to solve the MINLP defined in Chapter 3 and find those level parameters leading to the optimization goal:

Bibliography

[Ach+08] T. Achterberg, T. Berthold, T. Koch, and K. Wolter. *Constraint Integer Programming: a New Approach to Integrate CP and MIP*. eng. Tech. rep. 08-01. Takustr. 7, 14195 Berlin: ZIB, 2008 (cit. on p. 20).

[Ach09] T. Achterberg. "SCIP: solving constraint integer programs". In: *Mathematical Programming Computation* 1.1 (2009), pp. 1–41. URL: https://doi.org/10.1007/s12532-008-0001-1 (cit. on p. 20).

[Bes+23] K. Bestuzheva, A. Chmiela, B. MÃ¼ller, F. Serrano, S. Vigerske, and F. Wegscheider. *Global Optimization of Mixed-Integer Nonlinear Programs with SCIP 8*. Tech. rep. 2301.00587. arXiv, 2023 (cit. on p. 20).

[Chi+14] R. Chikhi, A. Limasset, S. Jackman, J. Simpson, and P. Medvedev. *On the representation of de Bruijn graphs*. en. arXiv:1401.5383 [q-bio.QM]. Oct. 2014. URL: http://arxiv.org/abs/1401.5383 (visited on 09/18/2026) (cit. on p. 7).

[Jai+20] C. Jain, A. Rhie, H. Zhang, C. Chu, B. P. Walenz, S. Koren, and A. M. Phillippy. "Weighted minimizer sampling improves long read mapping". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i111–i118. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i111/5870473 (visited on 09/18/2026) (cit. on p. 7).

[OS07] D. Okanohara and K. Sadakane. "Practical entropy-compressed rank/select dictionary". In: *2007 Proceedings of the ninth workshop on algorithm engineering and experiments (ALENEX)*. SIAM. 2007, pp. 60–70 (cit. on p. 12).

[Pib22] G. E. Pibiri. "Sparse and skew hashing of K-mers". en. In: *Bioinformatics* 38.Supplement_1 (June 2022), pp. i185–i194. URL: https://academic.oup.com/bioinformatics/article/38/Supplement_1/i185/6617506 (visited on 07/06/2026) (cit. on p. 7).

[SMR26] J. Schulte-Mattler and K. Reinert. "RSHash: a fast and space-efficient hash table for k -mers". en. In: *Bioinformatics* 42.Supplement_2 (Aug. 2026), btag440. URL: https://academic.oup.com/bioinformatics/article/doi/10.1093/bioinformatics/btag440/8767308 (visited on 09/15/2026) (cit. on pp. 5, 7, 10).

[ZKM20] H. Zheng, C. Kingsford, and G. Marçais. "Improved design and analysis of practical minimizers". en. In: *Bioinformatics* 36.Supplement_1 (July 2020), pp. i119–i127. URL: https://academic.oup.com/bioinformatics/article/36/Supplement_1/i119/5870484 (visited on 07/06/2026) (cit. on p. 6).